

Doc. No.: WG21/P0063R0

Date: 2015-09-25

Authors: Hans-J. Boehm, Clark Nelson

Email: hboehm@google.com

P0063R0: C++17 should refer to C11 instead of C99

We propose to have the C++ working document refer to C11 instead of C99. There appears to be little disagreement that this is fundamentally a good idea. The current situation creates several problems:

- C99 is not the current C standard. By 2017 implementations should support C++17 and C11. C99 will be increasingly irrelevant. Interoperation of C11 and C++17 will be what matters.
- By ignoring C11, we hide a number of important issues, including the interoperation of C11's threads and atomic operations APIs with their C++ counterparts. These issues are increasingly important to both users and implementers. Interoperability was a major design goal of the C++11 facilities, but it cannot be realized without implementation consistency in this area.
- Much more specifically and concretely, the effect of including `<stdalign>` is not well specified. The current claim is that it does substantially the same thing as including `<stdalign.h>`, but that header is described only in C11. (The same is not true of `<uchar>`. The `<uchar.h>` header is also new in C11 as compared with C99, but it was originally specified in the "new character data types" TR (19769), and for it the C++ standard refers to that specification.)

Tom Plum wrote a prior paper:

<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2013/n3631.pdf>

outlining changes in C11 and possible C++ impact.

Outline

This proposal has three parts:

- [The easy part: Replace the normative reference.](#)
- [Repair the resulting breakage.](#)
- [Optionally address C11 interoperability issues we've previously ignored.](#)

The easy part

Update 1.2 [intro.refs] p1.3:

(1.3) — ISO/IEC 9899:1999~~2011~~, Programming languages — C

Update 1.2 [intro.refs] p2:

(2) The library described in Clause 7 of ISO/IEC 9899:1999~~2011~~ and Clause 7 of ISO/IEC 9899:1999~~2011~~/Cor.1:2001~~2012~~ and Clause 7 of ISO/IEC 9899:1999~~2011~~/Cor.2:2003 is hereinafter called the C standard library.¹

Repair the resulting breakage

Since we generally don't blanket include C features, there seems to be little that is directly broken by the above change. The one issue we are aware of is:

Signal handlers

We need to make it clear that `thread_local`, and possibly other C11 extensions, remain unsupported in signal handlers. This will otherwise implicitly change when we refer to C11, since a POF ("plain old function") is defined in terms of C.

Richard Smith points out that the current POF definition has other issues. However, we believe its intent is generally reasonably understood. Failing to change the wording here would clearly mis-state our intent: Allowing `thread_local` in signal handlers was discussed by SG1 and rejected.

The signal handler definitions currently also rely in some subtle ways on the fact that atomic operations are function calls. We don't expect that to change, though particularly drastic approaches to accommodating C11 atomics might change it.

Address interoperability issues

Various C headers

As mentioned above, referring to C11, resolves a current issue with `stdalign.h`.

We are not required to say anything about other new C headers, since the supported ones are explicitly listed in 18.10 [support.runtime]. By default, we simply don't support others.

However, as we discuss below, in some cases silence leaves some significant practical issues unresolved.

Some minor issues are listed here. More major ones are listed below.

- There is no equivalent of `stdnoreturn.h`.
- The standard C++ library provides no facility for dynamically allocating memory with a specified alignment (`aligned_alloc`).
- The standard C++ library does not provide the `timespec` facility for querying fine-grained clock time.
- The `<cmath>` header does not define macros for `HAS_SUBNORM`, `TRUE_MIN`, or `DECIMAL_DIG` for each floating type.
- The `<cassert>` header does not define a macro named `static_assert` expanding to `_Static_assert`.

Optional components

C11 has several optional components, notably threads, atomics, and the annex K bounds-checking interfaces. C++ has traditionally resisted optional components. If we decide that we want to address interoperability with some of these components, we will have to address that issue.

Clause 29: Atomics

It would be immensely helpful for many of us to make the interaction with C11 atomics clear.

Here is a straw-man proposal approximately reflecting the current Android (since Lollipop) implementation:

`_Atomic` is not part of the C++ core language.

Including `cstdatomic` in C++ code has the effect of:

1. Including `<atomic>`
2. Promoting all definitions in the C header to global namespace
3. `#define _Atomic(T) atomic<T>`

Note that this effectively requires that C and C++ atomics be layout and ABI compatible, something we can't, and hopefully don't need to, require.

Clause 30: Threads

Do we want to say anything to enable interoperability of C threads and synchronization with C++ threads?

Do we want to somehow encourage C threads to be usable as C++ threads?

C and C++ don't agree on a common mutex type that could be used in a header included from both C and C++ code. Do we want to allow a `mtx_t` to be declared in C++ code? Should there be a wrapper object `c_mutex_t`, whose constructor takes a pointer to a C `mtx_t`, such that the resulting object meets C++ mutex requirements? (Perhaps we also need a `c_recursive_mutex_t`.)

Other C interoperability issues

The following were discovered while investigating this topic, but are not directly related to the move to C11:

The description of the `<stdio>` header is flawed. There is a table that mentions the names that are supposed to be defined and declared – but `<stdio.h>` from the C standard is not mentioned anywhere, so technically there is no specification of what any of those names is supposed to mean. Also, the table doesn't mention the `vscanf` function at all.

The description of `<cfenv>` is inconsistent with those of all the other C headers. It references the C standard by clause number instead of header name, and includes declarations for all the functions, in addition to taking them from the C standard by reference.